

T E C H N I C A L R E P O R T

ContractIQ

AI Contract Intelligence & Risk Analysis - Technical Report

Version 1.0 - API 0.5.0 - Frontend 0.6.0

Model: phase3-finetuned-minilm-squad2-1epoch (CUAD v1)

Stack: FastAPI - React - PyTorch - SQLite

September 2026

Abstract

ContractIQ is a local-first desktop web application that uses machine learning to review commercial contracts. A user uploads a PDF, DOCX or TXT file; the system parses it, extracts fifteen high-value clause types with a question-answering transformer fine-tuned on the CUAD v1 legal-contract dataset, links every finding to its exact character offsets in the source text, and computes a transparent, rule-based risk score on a 0-100 scale. The application is built as a React 19 single-page frontend over a FastAPI backend with SQLite persistence; the model runs locally on CUDA or CPU with no cloud calls and no external API keys.

On the official CUAD test set (102 contracts, evaluated once with frozen thresholds) the system achieves 60.8% exact match and 66.2% token F1 overall, with 74.4% accuracy on no-answer decisions, and a retrieval stage that places 95.2% of gold answers inside the model's search window. Quality is enforced by 199 automated tests, a verified evidence-offset invariant (0 mismatches across 129 checks), a 22-step end-to-end browser test with the real model, and a security review covering upload validation, path traversal and log privacy. This report documents the requirements, architecture, machine-learning pipeline, risk methodology, API design, verification programme, measured performance, and known limitations of the system.

Disclaimer: ContractIQ provides AI-assisted contract analysis and is not a substitute for professional legal advice. Results should be reviewed by a qualified professional.

Table of Contents

1. Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope	1
2. System Overview	2
2.1 Feature Summary.....	2
2.2 Design Principles	2
2.3 Technology Stack.....	2
3. System Architecture	3
3.1 Components	3
3.2 Request Lifecycle	3
3.3 Error Handling and Resilience	4
3.4 Data Model	4
4. Machine Learning Pipeline.....	5
4.1 Dataset	5
4.2 Task Formulation	5
4.3 Base Model Selection.....	5
4.4 Preprocessing and Windowing.....	5
4.5 Retrieval Optimization	6
4.6 Fine-Tuning	6
4.7 Evaluation Protocol and Results	6
4.8 Inference Design	7
4.9 Quantization Decision	7
4.10 Question Templates and Span Decoding.....	7
5. Risk Engine	8
5.1 Philosophy.....	8
5.2 Rule Catalog.....	8

5.3 Confidence Gate.....	8
5.4 Worked Example	8
6. Backend API	10
6.1 Endpoint Reference	10
6.2 Schemas and Error Envelope	10
6.3 Security Measures	10
6.4 Example Requests and Responses	11
7. Frontend.....	12
7.1 Pages.....	12
7.2 Design System and Signature Components.....	12
7.3 State, Data Fetching and Offline Tolerance.....	13
8. Deployment and Operations.....	14
8.1 Production Single-Port Serving	14
8.2 Launch Modes	14
8.3 Operations Notes.....	14
9. Quality Assurance	15
9.1 Test Inventory	15
9.2 Evidence Invariant	15
9.3 Defects Found and Fixed in Final QA	15
10. Performance.....	16
11. Limitations.....	17
12. Future Work	18
13. Conclusion	19
References	20
Appendix A. Environment and Reproduction.....	21
Appendix B. Third-Party Notices	22
Appendix C. Demo Contract Catalog.....	23

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Introduction

1.1 Motivation

Commercial contracts concentrate their most consequential terms - renewal mechanics, exclusivity, non-compete restrictions, liability caps, termination rights - inside long, repetitive documents. The CUAD v1 study reports a median contract length of roughly 6,700 tokens, with documents reaching 78,600 tokens, spread across 41 clause categories that legal teams must track. Manual review is therefore slow, expensive and error-prone, while cloud-based contract analysis tools raise a confidentiality problem: contracts are among the most sensitive documents an organisation holds, and sending them to third-party services is often unacceptable.

1.2 Problem Statement

Design and implement a system that runs entirely on a user's own machine, accepts a contract in PDF, DOCX or TXT form, reliably extracts the clauses that matter, shows exactly where in the document each finding comes from, and scores the contract's risk patterns with rules a non-expert can read and audit - without fabricating content and without pretending to be a lawyer.

1.3 Objectives

Extraction — Fine-tune a compact transformer for extractive question answering on the CUAD v1 dataset and package it for local inference on CUDA or CPU.

Evidence — Map every extracted clause to exact character offsets in the parsed document and verify that invariant end to end.

Risk — Compute risk with deterministic, documented heuristic rules, clearly separated from model confidence.

Robustness — Handle corrupt, encrypted, scanned and oversized files with typed errors instead of silent failures.

Product — Deliver a polished analysis workspace with history, search and recovery, plus a documented REST API.

Quality — Prove the above with automated tests, a real-model end-to-end run and a security review.

1.4 Scope

Version 1 targets single-user use on Windows, fifteen of the forty-one CUAD clause types, and synchronous analysis. Multi-user authentication, OCR for scanned documents and asynchronous job queues are explicit non-goals for this release and are discussed as future work.

2. System Overview

2.1 Feature Summary

ContractIQ accepts PDF, DOCX and TXT uploads, validates and sanitises them, parses the text with exact offsets, and analyses it with a fine-tuned MiniLM question-answering model covering fifteen clause types: Document Name, Parties, Agreement Date, Effective Date, Expiration Date, Renewal Term, Governing Law, Termination for Convenience, Non-Compete, Exclusivity, Anti-Assignment, License Grant, Audit Rights, Cap on Liability and Insurance. Each finding is presented with its source evidence highlighted in the document viewer, a confidence value, and the risk findings it triggered. Results persist to a searchable contract history with duplicate detection, and the engine status - including model state and compute device - is visible in the UI at all times.

2.2 Design Principles

Local-first. — uploads, inference and storage never leave the machine; there is no cloud dependency and no paid API.

Evidence-linked. — every AI-derived statement stores the exact character range it came from; the UI renders only from those offsets.

Transparent risk. — the risk score comes from fifteen readable rules, not from the model; AI confidence and risk are never conflated.

Honest uncertainty. — low-confidence extractions are flagged as uncertain and excluded from scoring rather than silently trusted.

Portable. — all paths are project-relative; the folder runs from any location, including paths with spaces.

2.3 Technology Stack

Layer	Technology	Version
Backend	Python, FastAPI, Uvicorn, SQLAlchemy, SQLite	3.11 / 0.128.0 / 0.40.0 / 2.0.49
Parsing	PyMuPDF, python-docx	1.26.7 / 1.2.0
Machine learning	PyTorch, Transformers, scikit-learn	2.13.0 / 5.16.1 / 1.9.0
Model	deepset/minilm-uncased-squad2, fine-tuned on CUAD v1	33.4M parameters
Frontend	React, TypeScript, Vite, Tailwind CSS, Framer Motion	19 / 5.8 / 7 / 4 / 12
Testing	pytest, vitest, Playwright	9.1.1 / 3.0 / 1.62

3. System Architecture

3.1 Components

The system is a strict four-layer pipeline: a React single-page application communicates over HTTP with the FastAPI backend; the backend orchestrates document parsing, machine-learning inference and rule evaluation; the ML layer is isolated behind a Python Protocol so the model can be replaced without backend changes; and SQLAlchemy persists everything to a single SQLite database file. All state changes flow in one direction - the UI never writes to the database directly.

Module	Responsibility
app/main.py	Application factory, request-ID middleware, CORS, error-envelope mapping, lifespan startup and crash recovery
app/api/	Route registration under /api; contracts, analyses, stats and health routers; dependency injection
app/core/	Environment-driven settings (CONTRACTIQ_* variables), typed exception hierarchy, structured logging
app/services/document_parser.py	PDF/DOCX/TXT extraction with offset fidelity; detects encrypted and text-less documents
app/services/contract_analysis.py	Upload, state machine, sha-256 duplicate detection, atomic persistence, idempotent re-analysis
app/services/ml_adapter.py	Lazy singleton wrapper exposing the analyzer behind a ClauseAnalyzer Protocol
app/risk/	Deterministic rule engine: 15 rules, weights, bands, confidence gate
app/db/	SQLAlchemy models and repository; transactions with rollback
ml/src/	Training, evaluation, retrieval, chunking, decoding and the ContractAnalyzer inference engine

3.2 Request Lifecycle

Upload: the file is validated (type, size up to 20 MB), stored under a UUID name, parsed to text with offsets, hashed (SHA-256 of normalised text) and persisted with status **READY**; a match on the hash flags the contract as a duplicate of an earlier upload. **Analysis:** the contract enters **ANALYZING**, the ML adapter lazy-loads the model, runs per-clause question answering over retrieval-selected text blocks, and returns spans with offsets; the risk engine evaluates its rules against those findings; results are written atomically and the status moves to **COMPLETED** (or **FAILED** with a typed error). The analyze operation is idempotent - re-analysis replaces the previous results - and a 409 is returned if an analysis is already in progress.

3.3 Error Handling and Resilience

Fifteen domain exceptions map to a single JSON envelope carrying an error code, a human message and a request ID; representative codes include `FILE_TOO_LARGE` (413), `CORRUPT_DOCUMENT` (422), `ENCRYPTED_DOCUMENT` (422), `OCR_REQUIRED` (422) and `MODEL_UNAVAILABLE` (503). A CUDA out-of-memory condition is caught and recorded as a failed analysis instead of crashing the server. On startup the application recovers contracts stuck in `ANALYZING` from a previous crash into a safe state. Contract text is never written to logs.

3.4 Data Model

Table	Key fields	Purpose
contracts	id (hex PK), filename, status, text_hash, raw_text, duplicate_of_id	Uploaded documents and parse state
analyses	contract_id FK, model_state, overall_risk_score/level, processing_ms	One row per analysis run
clause_results	analysis_id FK, clause_type, found, start_char/end_char, confidence, uncertain	Per-clause extraction with evidence offsets
risk_findings	analysis_id FK, rule_id, severity, weight, evidence, uncertain	Rule evaluations behind the score

4. Machine Learning Pipeline

4.1 Dataset

CUAD v1 (The Atticus Project) contains 510 US commercial contracts annotated with 41 clause categories, 20,910 question-answer pairs in total. The official split assigns 408 contracts to training and holds out 102 for testing; within the training portion, 367 contracts were used for fine-tuning and 41 for validation and threshold calibration. Dataset integrity is guarded by a recorded SHA-256 checksum of the raw archive.

4.2 Task Formulation

Clause extraction is cast as extractive question answering: each clause type has a natural-language question template (for example, "Is there a non-compete clause?"), the question and a block of contract text are tokenised together, and the model returns either an answer span or a no-answer decision. This formulation yields evidence for free - the answer span is the character range highlighted in the UI - and natively supports absence, which a pure classifier does not.

4.3 Base Model Selection

Candidate	Params	Throughput (CPU, win/s)	No-answer head	Decision
deepset/minilm-uncased-squad2	33.4M	4.43	Yes (SQuAD 2.0)	Selected
distilbert-base-uncased (SQuAD 1.1)	66M	3.41	No	Rejected: no no-answer training; slower
deepset/roberta-base-squad2	125M	~1.2	Yes	Rejected: 3.7x compute; tokenizer mismatch

4.4 Preprocessing and Windowing

Contracts far exceed the model's 512-token context, so documents are chunked into overlapping windows of 512 tokens with a stride of 128 and a maximum answer length of 64 tokens. The full training corpus expands to roughly 183,000 candidate windows; positive-answer windows are kept and negatives are sampled at a 1:3 ratio, yielding 20,420 windows for the fine-tuning run. Character-offset mappings are computed during tokenisation so decoded spans map back to exact document offsets.

4.5 Retrieval Optimization

On CPU, running every window of a long contract is the dominant cost, so a TF-IDF block-ranking stage (scikit-learn, local) selects the fifteen most relevant 1,200-character blocks per clause, boosted by clause-specific keyword lexicons and the document title. Measured gold-answer recall at K=15 improves from 79.4% (question only) to 91.4% (with keywords) to 95.2% (keywords plus title boost). Eleven of the fifteen clauses use retrieval; four high-value clause types (non-compete, exclusivity, cap on liability, license grant) still run over the full document because their recall demands it.

4.6 Fine-Tuning

Setting	Value
Run	1 epoch, 20,420 windows (5,105 positive / 15,315 sampled negative), batch 16
Optimiser	AdamW, learning rate 3e-5, weight decay 0.01, 10% warmup
Precision / seed	AMP fp16, seed 42
Hardware / wall time	NVIDIA RTX 2050 (4 GB), 1,277 steps in 838 seconds
Validation	token start accuracy 92.6%; thresholds frozen on the 41-contract dev split

4.7 Evaluation Protocol and Results

Thresholds were calibrated once on the validation split and frozen; the official 102-contract test set was then evaluated exactly once. Results are reported without post-hoc tuning.

Metric	Overall	Answerable (875 pairs)	No-answer (655 pairs)
Exact match (EM)	60.8%	50.6%	74.4%
Token F1	66.2%	60.2%	74.4% (accuracy)
Answerable detection	-	recall 83.2%	precision 81.2% (macro F1 82.2%)

Per-clause F1 spans a wide range, and the spread is reported rather than hidden: Document Name 89.0%, Governing Law 84.5%, Agreement Date 77.9%, License Grant 56.1%, and Parties 13.7%. Parties is the known weak spot - its gold answers frequently span multiple entity mentions across the preamble, which the single-span decoder cannot fully capture. This is disclosed in the UI and flagged as future work (multi-span decoding).

4.8 Inference Design

The backend wraps a single ContractAnalyzer instance behind a ClauseAnalyzer Protocol, lazy-loaded on first analysis. The final fine-tuned checkpoint ships inside the repository (ml/models/final, ~133 MB); if it is missing the adapter falls back to the public baseline and reports `model_state = zero_shot_baseline` honestly. Compute device is detected at startup (CUDA when available, CPU otherwise), and a CUDA out-of-memory error during analysis is contained as a failed analysis rather than a process crash.

4.9 Quantization Decision

INT8 dynamic quantization was benchmarked and rejected: it delivered only a 1.24x speed-up while flipping 56% of argmax span decisions - an unacceptable accuracy trade for a system whose core promise is evidence fidelity.

4.10 Question Templates and Span Decoding

Each enabled clause type carries a curated question template from CUAD, for example: "Is there a non-compete restriction?" or "What is the Cap on Liability?". At inference the question is tokenized with each candidate text block; the model produces a start-logit and end-logit for every token, and the decoder scores all plausible spans (start before end, within the 64-token limit) by $\text{start_logit} + \text{end_logit} - \text{length penalty}$. The highest-scoring span competes against the no-answer score (the logits at the [CLS] position); a span is only returned when its score beats the no-answer score by more than the calibrated threshold. Calibrated per-split thresholds were frozen on the validation split: a span that wins is emitted with its character offsets and confidence; otherwise the clause is reported as absent with the no-answer confidence. This is why the system can be wrong in only two visible ways - extracting the wrong span or declaring absence - and both are visible in the UI with their evidence.

5. Risk Engine

5.1 Philosophy

The risk score is deliberately not a machine-learning output. It is the sum of weighted, human-readable rules evaluated against extraction findings, capped at 100, and banded LOW (0-30), MEDIUM (31-60) and HIGH (61-100). Because the rules live in a configuration file (ml/configs/clauses.json), weights and bands are data that can be reviewed and tuned without touching code. The maximum reachable score under the current rule set is 89.

5.2 Rule Catalog

Rule	Weight	Severity
NON_COMPETE_PRESENT	18	HIGH
EXCLUSIVITY_PRESENT	15	HIGH
CAP_ON_LIABILITY_ABSENT	12	MEDIUM
AUTO_RENEWAL_LANGUAGE (regex)	10	MEDIUM
TERMINATION_CONVENIENCE_ABSENT	10	MEDIUM
ANTI_ASSIGNMENT_RESTRICTIVE	8	MEDIUM
RENEWAL_TERM_PRESENT	5	LOW
INSURANCE_REQUIREMENT_PRESENT	4	LOW
AUDIT_RIGHTS_PRESENT	4	LOW
ANTI_ASSIGNMENT_PRESENT	3	LOW
5 informational rules (parties, dates, governing law, license, document name)	0	INFO

5.3 Confidence Gate

A rule only contributes its weight when the underlying extraction confidence is at least 0.5; below that threshold the finding is marked uncertain, shown distinctly in the UI, and excluded from the score. This keeps the number stable against weak model output and is the concrete mechanism separating AI confidence from risk.

5.4 Worked Example

A 3,649-character master services agreement with non-compete, exclusivity and evergreen-renewal language scores 67 (HIGH): +18 non-compete, +15 exclusivity, +10 auto-renewal, +8 anti-assignment, +5 renewal term, +4 insurance, +4 audit rights and +3 anti-assignment. Every contribution is listed with its evidence, so a reviewer can contest any single line item.

The full finding list from that analysis, exactly as stored and displayed:

Rule fired	Weight	Severity	Extraction confidence
NON_COMPETE_PRESENT	18	HIGH	0.85
EXCLUSIVITY_PRESENT	15	HIGH	0.88
AUTO_RENEWAL_LANGUAGE (regex)	10	MEDIUM	0.97
ANTI_ASSIGNMENT_RESTRICTIVE	8	MEDIUM	0.98
RENEWAL_TERM_PRESENT	5	LOW	0.97
INSURANCE_REQUIREMENT_PRESENT	4	LOW	0.81
AUDIT_RIGHTS_PRESENT	4	LOW	0.98
ANTI_ASSIGNMENT_PRESENT	3	LOW	0.98
PARTIES_IDENTIFIED (uncertain)	0	INFO	0.15 - excluded from score

6. Backend API

6.1 Endpoint Reference

Method	Path	Purpose
GET	/api/health	Service, database and model status (state, device, version)
POST	/api/contracts/upload	Multipart upload; validate, parse, persist (201)
GET	/api/contracts	List with search, status/risk filters, sorting, pagination
GET	/api/contracts/{id}	Contract metadata plus latest analysis summary
GET	/api/contracts/{id}/text	Raw parsed text for evidence highlighting
POST	/api/contracts/{id}/analyze	Run extraction and risk scoring; idempotent; 409 if busy
GET	/api/contracts/{id}/analysis	Full latest analysis: clauses, evidence, findings
DELETE	/api/contracts/{id}	Delete contract, analyses and stored file
GET	/api/analyses/{id}	Fetch a specific analysis
GET	/api/stats	Dashboard totals, risk distribution, average processing time

6.2 Schemas and Error Envelope

Responses are Pydantic-validated. Every error returns a consistent envelope {error, message, request_id}; the request ID is also attached as an X-Request-ID response header by middleware, which makes support and log correlation straightforward. The frontend maps error codes to friendly, actionable messages.

6.3 Security Measures

Upload validation: — extension and MIME checks, 20 MB size cap, text re-parse validation; corrupt, encrypted and scanned files are rejected with typed errors.

Path safety: — user-supplied filenames are never used as paths; uploads are stored as UUID names; filenames are sanitised against traversal.

Log privacy: — contract text is never logged; logs carry structured events and IDs only; CORS is restricted to the local frontend origin.

Data minimisation: — raw extracted text is stored only when enabled (default on, needed for highlighting), with a documented toggle; temp files are cleaned on a 24-hour retention policy.

6.4 Example Requests and Responses

Upload (multipart, field name upload):

```
POST /api/contracts/upload (file=demo.docx) -> 201
{ "id": "6e60c2ff...", "filename": "demo.docx", "status": "READY",
  "character_count": 3649 }
```

Analyze (synchronous; the same call is safe to repeat):

```
POST /api/contracts/6e60c2ff.../analyze -> 200
{ "analysis_id": 2, "contract_id": "6e60c2ff...", "status": "COMPLETED" }
```

Fetch the full analysis (abridged response):

```
GET /api/contracts/6e60c2ff.../analysis -> 200
{ "overall_risk": { "score": 67, "level": "HIGH" },
  "clauses": [ { "clause_type": "non_compete", "found": true, "confidence": 0.85,
    "start_char": 1740, "end_char": 2086, "uncertain": false }, ... ] }
```

A typed failure looks like:

```
{ "error": "OCR_REQUIRED", "message": "The PDF has no text layer...", "request_id":
  "dbb1496526d944b5" }
```

7. Frontend

7.1 Pages

Route	Page	Content
/	Overview	Hero, portfolio stats, risk constellation ring, recent activity
/analyze	New Analysis	Drag-and-drop upload, honest staged progress animation
/contracts	History	Search, filters, sorting, pagination over past contracts
/contracts/:id	Analysis Workspace	Document viewer with evidence highlights, Risk Orb, clause map, findings
/intelligence	Intelligence	Model status, compute device, risk methodology, disclaimer

7.2 Design System and Signature Components

The interface follows a "Fluid Legal Intelligence" design language: an ink-dark base with ivory text, mint/aqua primaries and coral reserved for high risk; serif display typography; glass surfaces and an animated aurora background that respects reduced-motion preferences. Three components carry the experience. The Risk Orb is an animated segmented arc with a numeric readout (never colour alone, exposed with role="img" for assistive technology). The Clause Intelligence Map is a signature SVG showing the contract as a hub with its fifteen clause nodes, keyboard-accessible. The DocumentViewer renders evidence highlights from stored character offsets using highlight-scoped rendering, so even a 197,000-character document produces only a handful of DOM nodes.

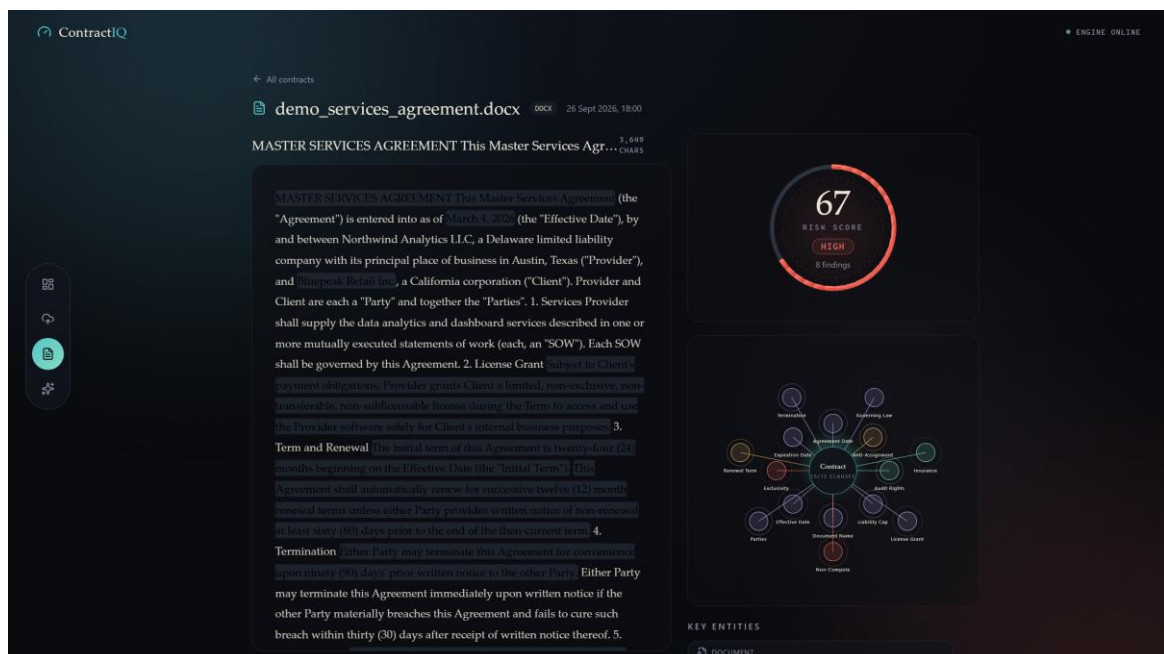


Figure 1. Analysis workspace: evidence highlighting, Risk Orb (67 - HIGH) and Clause Intelligence Map.

7.3 State, Data Fetching and Offline Tolerance

State is deliberately plain React hooks - no external store. The API client attaches per-endpoint timeouts via AbortController (15 s fast paths, 60 s uploads, 300 s analyses) and translates typed backend errors into friendly copy. A health hook polls engine status every 30 seconds while online and every 12 seconds while offline, with a manual retry; the UI remains navigable and shows queued-state feedback when the backend is unreachable.

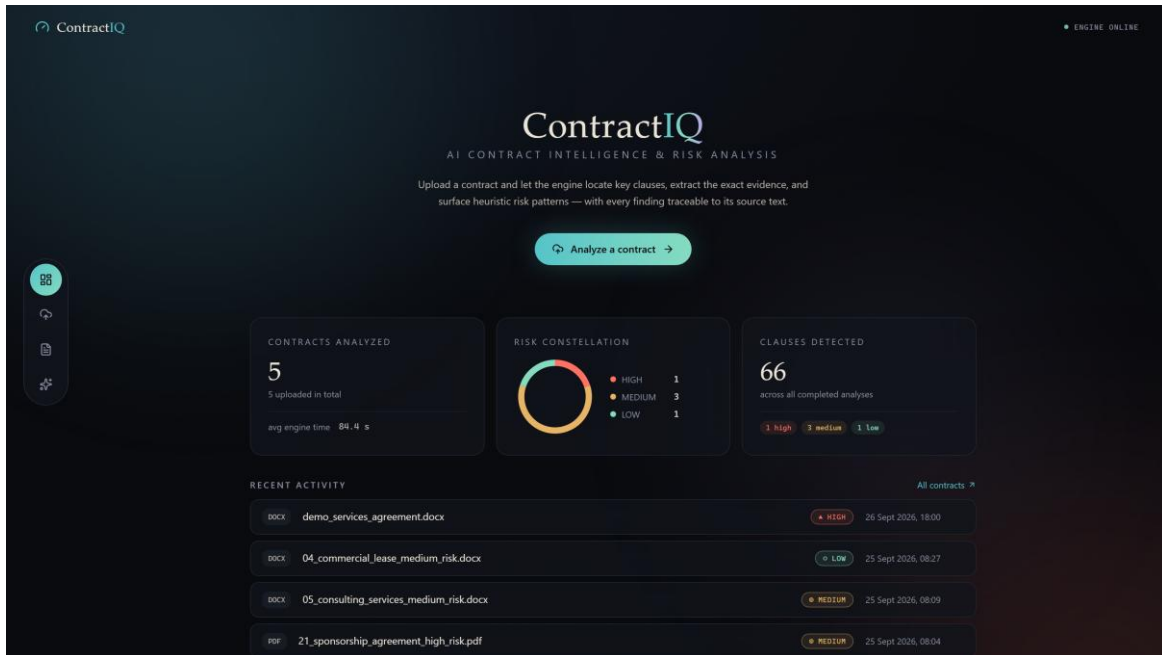


Figure 2. Overview dashboard with portfolio statistics and engine status.

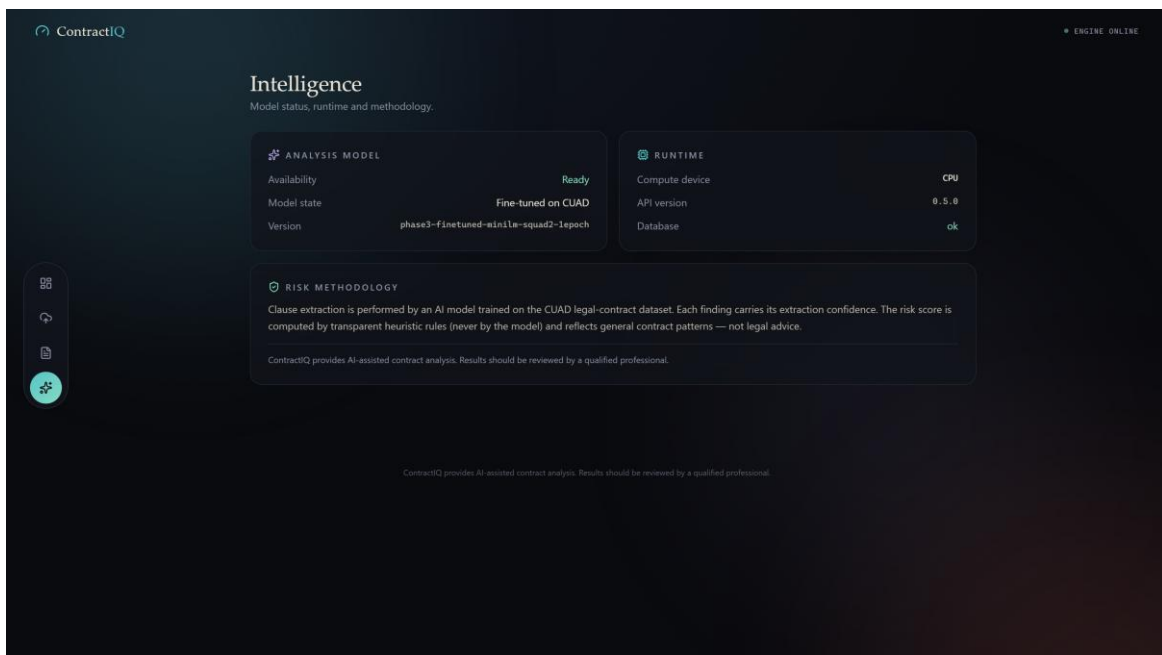


Figure 3. Intelligence page: model state, compute device and risk methodology shown to the user.

8. Deployment and Operations

8.1 Production Single-Port Serving

The deployed configuration serves the built React frontend and the API from one FastAPI process. When frontend/dist exists (produced once with `npm run build`), the application mounts it: hashed assets are served statically and every non-API GET falls back to `index.html` so single-page deep links such as `/contracts/<id>` resolve on refresh. Unknown `/api` paths still return JSON 404s. The production frontend addresses the API with a same-origin base (`/api`), which makes the built bundle host-agnostic - the identical build works on localhost, a LAN IP, or any port without rebuilding.

8.2 Launch Modes

Launcher	Binding	Use
<code>run_contractiq_prod.bat</code>	127.0.0.1:8010	local single-user use; nothing exposed to the network
<code>run_contractiq_lan.bat</code>	0.0.0.0:8010	demo to phones/other laptops on trusted Wi-Fi (no auth - documented caveat)
<code>run_contractiq.bat</code>	dev servers	development: Vite hot reload on :5173 + API on :8000

The default production port is 8010 (override via the `BACKEND_PORT` environment variable). Port 8000 is deliberately avoided as a default because another active project on the development machine occupies it - a collision that repeatedly killed servers during bring-up and motivated both the port choice and the same-origin API design.

8.3 Operations Notes

Startup: — the server starts in ~10 s; the ML model lazy-loads on first analysis (~30 s once), after which analyses run in seconds (GPU) to a couple of minutes (CPU).

State: — state lives entirely in storage/ (SQLite database, uploads, temp). Copying that folder moves the full history; deleting it resets the app.

Portability: — the folder was verified to run from a fresh copy at a path containing spaces, with zero hardcoded machine paths (150-file scan).

Driver mismatch: — with the CUDA torch build installed but an older NVIDIA driver present, the app detects the mismatch and serves on CPU instead of crashing (Section 4.8).

9. Quality Assurance

9.1 Test Inventory

Suite	Coverage	Result
pytest (backend, ML, integration)	Parsers, risk engine, repository, API contract, ML decoding	186 passed
vitest (frontend)	Components and evidence-offset helper logic	13 passed
Production build and lint	Vite build, ESLint, TypeScript strict	Clean
Browser E2E (Playwright)	22-step real-model flow: upload, analyze, verify	Passed, 0 page errors

9.2 Evidence Invariant

The defining correctness property - that every stored clause span exactly matches the text at those offsets in the served document - was checked across 129 verifications spanning browser checks and database rows from real analyses, with zero mismatches.

9.3 Defects Found and Fixed in Final QA

The final QA phase surfaced three high-severity defects, all fixed and covered by regression tests: a transaction rollback that swallowed a FAILED status update (leaving contracts stuck as ANALYZING), a foreign-key violation when deleting a contract with analyses under SQLite enforcement, and a missing import on the 409 analysis-in-progress path that raised an unhandled exception.

10. Performance

Measurement	Result
Model throughput, GPU (RTX 2050, batch 8)	52.8 windows/s - 11.9x the CPU baseline (4.43 windows/s)
End-to-end analysis time	~4-12 s per contract on GPU; ~80 s median on CPU
Largest live test (197,160 characters)	16.9 s on GPU
Upload round-trip / health check	159 ms / 29 ms
Cold start to ready	16 s including model load
Memory stability	VRAM flat at 0.141 GB; 0.0 MB RSS growth over 50 requests
Frontend production bundle	135 kB gzipped

11. Limitations

Single user: — the application is intentionally single-user and local; there is no authentication layer.

No job queue: — analysis is synchronous; the client waits for completion rather than streaming progress from a queue.

Heuristic risk: — risk rules encode expert heuristics, not patterns learned from dispute outcomes.

No OCR: — scanned PDFs require OCR, which is not implemented; they are rejected with a typed error.

Parties weak spot: — multi-entity party spans extract poorly (13.7% F1) under the single-span decoder.

12. Future Work

Async pipeline. — move analysis to a background queue with live progress streaming to the UI.

OCR support. — add a Tesseract-based OCR stage so scanned contracts enter the same workflow.

Learned risk layer. — learn rule weights or a small risk model on top of extracted clause features.

Wider coverage. — enable further CUAD clause types (a configuration flag away) and add multi-span decoding for parties.

Team mode. — add authentication and a multi-user mode for small teams.

13. Conclusion

ContractIQ demonstrates that a focused, well-evaluated extractive QA model - small enough to run on a laptop - can power a genuinely useful contract-review tool when it is wrapped in the engineering that makes its outputs trustworthy: exact evidence offsets, typed failure modes, transparent scoring, and a verification programme that tests the whole chain with the real model. The system is complete against its Phase 1-9 plan with no release blockers, and its honest reporting of weak spots (parties extraction, heuristic risk) defines a concrete, tractable roadmap for the next iteration.

References

- [1] M. Hendrycks, D. Burns, et al., "CUAD: A Comprehensive Legal Contract Review Dataset," The Atticus Project, 2021. <https://www.atticusprojectai.org/cuad>
- [2] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," NeurIPS, 2020.
- [3] deepset, "minilm-uncased-squad2," Hugging Face model card. <https://huggingface.co/deepset/minilm-uncased-squad2>
- [4] P. Rajpurkar, R. Jia, and P. Liang, "Know What You Don't Know: Unanswerable Questions for SQuAD," ACL, 2018.
- [5] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," EMNLP System Demonstrations, 2020.
- [6] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," NeurIPS, 2019.
- [7] F. Ramírez, "FastAPI" documentation. <https://fastapi.tiangolo.com>
- [8] SQLAlchemy 2.0 documentation. <https://docs.sqlalchemy.org>
- [9] Meta Open Source, "React" documentation. <https://react.dev>
- [10] E. Youn and Y. Cremer, "Vite" documentation. <https://vite.dev>

Appendix A. Environment and Reproduction

Requirements: Windows 10/11, Python 3.11-3.13, Node.js 20+ (tested on Node 24).
Optional: NVIDIA GPU with an up-to-date driver for CUDA inference; CPU-only machines are fully supported.

Automated setup and launch

```
setup_windows.bat (creates .venv, installs all dependencies, detects GPU)
run_contractiq.bat (starts backend :8000 and frontend :5173, opens the browser)
stop_contractiq.bat (stops both)
```

Test commands

```
.venv\Scripts\activate
python -m pytest backend\tests ml\tests -q
cd frontend && npm run test && npm run lint && npm run build
```

Key configuration (all optional, env-prefixed CONTRACTIQ_*)

```
CONTRACTIQ_DB_URL, CONTRACTIQ_UPLOAD_DIR, CONTRACTIQ_TEMP_DIR,
CONTRACTIQ_MAX_UPLOAD_MB (default 20), CONTRACTIQ_STORE_RAW_TEXT (default true),
CONTRACTIQ_CORS_ORIGINS, CONTRACTIQ_RISK_MIN_CONFIDENCE (default 0.5)
```

Appendix B. Third-Party Notices

Component	License
CUAD v1 dataset	CC-BY-4.0 (The Atticus Project)
deepset/minilm-uncased-squad2	CC-BY-4.0 (attribution included in ml/models/final)
PyTorch	BSD-3-Clause
Hugging Face Transformers	Apache-2.0
scikit-learn	BSD-3-Clause
FastAPI, Uvicorn, SQLAlchemy	MIT
React, Vite, Tailwind CSS	MIT
PyMuPDF	AGPL-3.0 / commercial dual license
python-docx	MIT

Full attribution text is reproduced in `THIRD_PARTY_NOTICES.md` in the project root. The project itself carries no redistribution license; all rights are reserved by the author.

Appendix C. Demo Contract Catalog

The project ships a presentation kit of 22 synthetic sample agreements (demo_contracts/, DOCX with 5 PDF samples) spanning LOW / MEDIUM / HIGH risk profiles. The folder is presentation material only - the application never reads it, and files enter the app only when manually uploaded. The catalog is generated by demo_contracts/generate_demo_contracts.py and can be extended or regenerated without touching the application.

File	Agreement type	Intended risk story
01	SaaS subscription (.docx + .pdf)	LOW - friendly terms, cap and insurance present
02	Mutual NDA	LOW - minimal obligations
03	Freelance contractor	LOW - short term, easy exit
04	Commercial lease	MEDIUM - auto-renewal, assignment limits
05	Consulting services	MEDIUM - auto-renewal, soft non-compete
06	Exclusive supply / vendor	MEDIUM - exclusivity, auto-renewal
07	Executive employment (.docx + .pdf)	HIGH - non-compete, no convenience exit
08	Exclusive distribution (.docx + .pdf)	HIGH - non-compete, purchase minimums
09	Franchise (.docx + .pdf)	HIGH - lock-in across most rules
10	LLC operating agreement	MEDIUM - member restraints
11	Loan / promissory note	LOW - standard bank terms
12	Enterprise software license	MEDIUM - auto-renewal, audit
13	Joint venture (.docx + .pdf)	HIGH - non-compete, tech exclusivity
14	Non-exclusive reseller	LOW - easy exit
15	Real-estate purchase	LOW - one-shot, capped liability
16	Content creator / influencer	MEDIUM - category exclusivity
17	Toll manufacturing	MEDIUM - capacity commitment
18	University research collaboration	LOW - academic terms
19	Equipment rental	MEDIUM - auto-renewal, no cap
20	GDPR data processing	LOW - protective, capped
21	Event sponsorship (.docx + .pdf)	HIGH - category exclusivity, no late exit
22	Logistics & warehousing	MEDIUM - auto-renewal, low cap